Selinna Lin

6.1210 MIDTERM 1 CRIB SHEET

Sterling's Approximation: $n! = \Theta(\sqrt{n}(\frac{n}{e})^n)$

Log Rules:

- $\log_b(M^k) = k\log_b(M)$
- $\log_b(1) = 0$
- $\log_b(b) = 1$
- $\log_b(b^k) = k$
- $b^{\log_b(k)} = k$
- $\log_b(a) = \frac{\log(a)}{\log(b)}$

Geometric Series:

n^{th} term: $a_n = ar^{n-1}$ n: # of terms

sum: $\frac{a(1-r^n)}{1-r}$ OR $\frac{a(r^n-1)}{r-1}$ if $r \neq 1$

Infinite: $S = \frac{a_1}{1-r}$ $(r<1)$

FLOPS: (Floating point operations per second)

kilo: 10^3	giga: 10^9	peta: 10^{15}	zetta: 10^{21}
mega: 10^6	tera: 10^{12}	exa: 10^{18}	yotta: 10^{24}

Recurrences:

$T(n) = aT(\frac{n}{b}) + \Theta(n^c)$ a: # of subproblems

# of levels: $\log_b n$

# of nodes at level i: a^i

work done at each node: $\Theta((\frac{n}{b^i})^c)$

work done at level i: $a^i * \Theta((\frac{n}{b^i})^c)$

work done for $\log_b n$ levels: $\sum_{i=0}^{\log_b n - 1} a^i * \Theta((\frac{n}{b^i})^c)$

$\sum_{i=0}^{\log_b n - 1} a^i = \frac{a^{\log_b n} - 1}{a-1}$

Master Theorem:

$T(n) = aT(\frac{n}{b}) + f(n)$

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & f(n) = O(n^{\log_b a - \varepsilon}) \\ \Theta(n^{\log_b a}\log n) & f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & f(n) = \Omega(n^{\log_b a + \varepsilon}) \\ & \text{and } af(\frac{n}{b}) < cf(n) \text{ for larger } n. \end{cases} \quad \varepsilon > 0,\ c < 1$$

Runtime for Sorting:

- Insertion $O(n^2)$
- Selection $O(n^2)$
- Merge $O(n\log n)$
- Counting $O(n+u)$
- Radix $O(n + n\log_n u)$

Interface:
- Stack (LIFO)
- Queue (FIFO)

Dynamic Array: Double/halve when too full/empty.
- insert/delete amortized time
n inserts of $O(1)$ + double $O(n) \to O(1)$ am

Sorting:

Stable: Order of equal elements is preserved

In-place: Amount of memory used beyond storing input is $O(1)$.

Insertion Sort

★ Runtime: $\Theta(n^2)$
★ Inplace ★ Stable

*Good when mostly sorted.

Ex: [20,17,19,13,2,5,4]
[20,17,19,13,2,5,4]
[17,20,19,13,2,5,4]
[17,19,20,13,2,5,4]
[13,17,19,20,2,5,4]
[2,13,17,19,20,5,4]
[2,5,13,17,19,20,4]
[2,4,5,13,17,19,20]

Selection Sort

★ Runtime: $\Theta(n^2)$
★ In-place ★ Not stable

Ex: [20,17,19,13,2,5,4]
[17,19,13,2,5,4,20]
[17,13,2,5,4,19,20]
[13,2,5,4,17,19,20]
[2,5,4,13,17,19,20]
[2,4,5,13,17,19,20]
[2,4,5,13,17,19,20]
[2,4,5,13,17,19,20]

Merge Sort

★ Runtime: $\Theta(n\log n)$
★ not in-place ★ Stable

- Recursively sort first & second half
- Merge both lists by interleaving one element at a time & comparing the two.

Comparison-based model: $\Omega(n\log n)$

Worst-case Analysis: (opposite for best-case)
- Upper Bound → A has worst case runtime $O(f(n))$ if $\exists c, n_0$ s.t. $\forall n \geq n_0$, $\forall x$ of length n, algorithm A takes $\leq cf(n)$ steps on input x (all)
- Lower Bound → $\Omega(f(n))$... $\forall n \geq n_0, \exists x$ of length n s.t. A takes $\geq cf(n)$ steps on input x (one)

Counting Sort

$u = O(n)$

* Good for if you know range
★ Runtime: $\Theta(n+u)$
★ Not in-place ★ Stable

1. DAA sort but with chaining
2. If chain with queue → stable

Radix Sort

★ Runtime: $\Theta(n + n\log_n u)$

Tuple sorting by using counting sort.

1. Write all keys $x.k$ in base n.

$x.k = k_1 n^{c-1} + k_2 n^{c-2} + \ldots + k_c$

↓ creates a tuple

$(k_1, k_2, \ldots, k_c)$

need $n^c \geq u$ $c = \lceil \log_n u \rceil$

Note: New key same as old, just in a new form

Ordering on u = lexicographical ordering on $(k_1, \ldots, k_n)$

2. To sort lexicographically, use tuple sort.
3. In tuple sort, use counting sort as the internal sort.

Tuple Sort

Sort lexicographically

Heap Sort

Build max heap from unsorted array.

- Find max element A[0] ②
- Swap elements A[0] and A[n-1] → Brings max to end of array
- Discard A[n-1] from heap → takes max out of heap, makes heap one smaller
- Reheapify root $O(\log n)$
- Go back to 2 until heap empty. Total: $O(n\log n)$

Selinna Lin

Binary Search Tree (BST):

Useful for:
- kth smallest element
- inserting/deleting
- find_min / find_max
- find_prev / find_next } Hash table bad at this

Algorithms:

▶ traverse(node) O(n)
```
if node == None:
    return
else:
    traverse(node.left)
    print(node)
    traverse(node.right)
```
(in-order traversal)

▶ find_key(k): binary search O(h)

▶ insert(k): traverse using binary search to see where it fits O(h)

▶ find_min(): left most leaf (keep going left) O(h)

▶ successor(x): O(h)
If x has right child → find_min(x.right)
no right child → no parent → no successor
→ x left child → parent is succ.
→ x right child → keep going up ancestors until one of ancestors is a left child

AVL Trees:

- In-order traversal maintained through rotations

Augmentations:
- size: if none, size is 0
  else, x.size = x.left.size + x.right.size + 1
- height: if leaf, height is 0
  else, x.height = 1 + max(left.h, right.h)

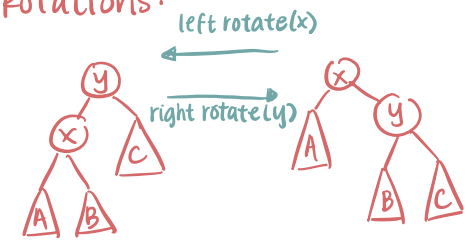
Updating → only need to update nodes on path from x to root

- AVL property: $|skew| \le 1$
  $|skew| = |left.height - right.height| \le 1$

Tree has height $h = \Theta(\log n)$

Case 1: y is right-heavy → left-rotate(x) O(1)
Case 2: y is balanced → left-rotate(x) O(1)
Case 3: y is left-heavy → right-rotate(y), left-rotate(x)

Rotations:

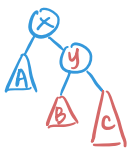
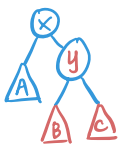
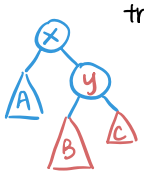


"Rotate the root ? in the ? direction"

{A, x, B, y, C} {A, x, B, y, C}

* Note: Rotations maintain same in-order traversal.

How to balance: If x is "right-heavy":

1. 
2. 
3. 

*Use after insert/delete mess up the skew

Heaps: implements a priority queue

Heap Property: For all x, x.k > x.left.k and x.k > x.right.k (max)

Heaps as a tree:
root → i = 1
parent(i) → i/2
left(i) → 2i
right(i) → 2i+1
} no pointers!

Heaps	vs.	BST:
- Pointerless		- Pointers
- Good for delete min/max		- Good for find prev/next
- Not sorted		- Sorted

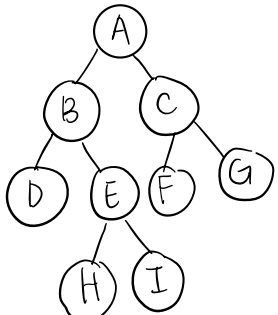
Runtimes:

build_max_heap	O(n)
max_heapify	O(log n)
insert	O(log n)
delete_max	O(log n)
heap_sort	O(n log n)

Height of heap: $h \le \lceil \log(n) + 1 \rceil$

max_heapify_down(i): used for deletion
delete_max(): swap greatest node with rightmost leaf, heapify-down (swapping with greater child) O(log n)

max_heapify_up(i): used for insertion



in-order traversal:
D, B, H, E, I, A, F, C, G

pre-order: Return nodes in order of visit
A, B, D, E, H, I, C, F, G

post-order: Return all the descendants of node visited before returning the node.
D, H, I, E, B, F, G, C, A

Hashing:

$\alpha = \frac{\#\text{ of elements}}{\text{size of hash table}}$

Chaining → For keys a, b, h(a) = h(b), we can chain values.
Hash functions → Want chain to be evenly distributed

SUHA: Each key k maps to a uniformly random h(k) in $[0, \cdots, m-1]$, independent of other keys.

load factor has to be constant

load factor: $E[\text{size of chain @ } h(k)] = \frac{1}{m}\sum_{i=1}^{m}(\text{size of chain @ index } i) = \alpha = \frac{n}{m}$

* If $\alpha = \frac{n}{m} \gg 1$, rebuild hash table for bigger size
* "Find" then takes $O(1 + \frac{n}{m})$ time.

"Average-case" probably implies hashing

Amortization:
"Amortized O(1) time" means doing the operation k times takes O(k) time.

O(1) excepted time to find item.
If all elements hash to same value, (chain of length n) it can take O(n) time to find item.

Insert + delete take O(1) time for average case.
Worst case O(n).

6.1210 MIDTERM 2 CRIB SHEET

Selinna Lin

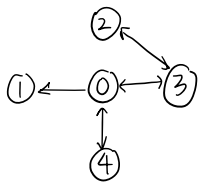
Graphs:

- $G=(V,E)$
- $n=|V|$
- $m=|E| = O(n^2)$

"Path": Sequence of possibly repeating vertices

"simple path": path with no repeating vertices.

Graph Representations:



Vertices i point to Adjacency list: (+)
0 [1, 3, 4]
1 []
2 [3]
3 [0, 2]
4 [0]

Adjacency Matrix:

	0	1	2	3	4
0	0	1	0	1	1
1	0	0	0	0	0
2	0	0	0	1	0
3	1	0	1	0	0
4	1	0	0	0	0

Adjacency Set:
0: {1, 3, 4}
1: { }
2: {3}
3: {0, 2}
4: {0}

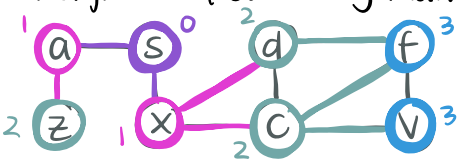
Adjacency List:
list of index corresponding to a vertex to the list of neighbors of that vertex.
+Good for sparse graphs $|E|=O(n)$
more space economical

Adjacency Matrix:
$|V|\times|V|$ entry $(i,j)=1$ if (i,j) is an edge on graph, 0 otherwise
+Good for dense graphs $|E|=O(n^2)$
Faster edge check

Adjacency Set:
DAA (Direct Access Array) mapping each vertex to the set of neighbors.

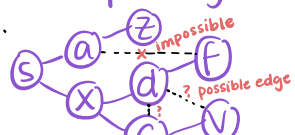
Breadth-First Search (BFS): Runtime: $O(|E|+|V|)$

1. Start at node s.
2. Until no new vertices are marked, mark all neighbors of currently marked vertices.



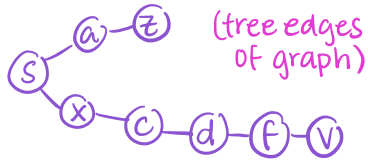
BFS Use: Shortest Path (unweighted edges)
Reachable Nodes

BFS Spanning Tree:



level indicates shortest path from root!

DFS Spanning Tree:



(tree edges of graph)

Depth-First Search (DFS): Runtime: $O(|V|+|E|)$

1. Start at vertex s.
2. List all its neighbors. Then all their neighbors.
   ⋮
   etc.

DFS Uses: Find cycles
Find reachable nodes
Topological sort
back edge ⟺ cycle

DFS-Spanning Tree: (helpful for directed graphs)

Edges:

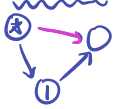
tree edge (to children)
* only type of edge part of DFS-tree

g→w
g→g
g→b

back edge (descendant)

w→g
g→g
b→g

forward (ancestor)



g→w
g→g
g→b

cross (unrelated subtree)

w→g
w→b
g→b

(w→w & b→b can be any edge).

Note: Undirected graphs → no forward/cross edges.

* Note: Doesn't work for cyclic graphs.

Topological Sort: Find order in which to relax DAG.

Numbering the vertices of a DAG s.t. all edges go from low→high vertex numbers.

- Print vertices according to finish times DFS

ABCCBDEEDA
↑↑ ↑ ↑↑
finishing times. → CBEDA

* Topological sort IFF DAG!

- Take reverse finish order: ADEBC

* SCC & WCC for directed graphs

Kosaraju-Sharir SCCs Algorithm: Find SCCs & make condensation graph

1. Take G_{rev} & initialize leader[v] ≠ for all v in G.
2. 1st Pass → Run full DFS on G_{rev} & record finish times
3. 2nd Pass → For each vertex in reverse finish time,
   - Set current = v
   - If v is unvisited, run DFS starting from v & set leader[u]=v if leader[u]≠,
   - Return leaders array

H has one vertex for every SCC of G.
∃ edge (C_1,C_2) if edge (u,v) in G. u ∈ C_1, v ∈ C_2

Runtime: $O(|V|+|E|)$

SCCs with >1 vertex ⟺ cycle

DAG Relaxation: Finds shortest path in DAG.

1. Find topological order.
2. Relax each edge once in topological order.

```
def DAG_sp(G,s):
    d[v] = ∞
    d[s] = 0
    for u in topo_sort(V):
        for (u,v) in E:
            try_to_relax((u,v))
    return d
```

Runtime: $O(|V|+|E|)$

* DAG Relaxation works with negative weighted edges.

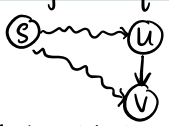
Relaxation:

```
def try_to_relax(u,v):
    if d(u)+w(u,v) < d(v):
        d(v) = d(u)+w(u,v)
        parent(v) = u
```

(faster route to v through u)

Safety Lemma:
Relax preserves the invariant $d[v] \geq \delta[v]$ $\forall v \in V$.

Triangle Inequality:



$\delta(v) \leq \delta(u) + w(u,v)$

Optimal Subproblem Property:

If p is shortest path from s→v, then a segment of p is also shortest path from s→u.

Selinna Lin

Dijkstra's Algorithm:
Finds shortest path for nonnegative weighted edges.

```
def dijkstra(G,s):
    ∀v, d[v]=∞, Parent[v]=ϕ
    d[s]=0
    T={s}                      ← min Priority queue
    pq=PQ.build(d)
    while pq:                  (while pq not empty)
        v1 = pq.delete_min()
        T.add(v1)
        for (v1,v2) ∈ E:
            relax(v1,v2)
            pq.update(v2)
    return d, parent
```

* Nodes with shorter distances get dequeued first, and nodes with longer distances get dequeued later.

Runtime: $O(|V|\log|V| + |E|)$

Bellman-Ford Algorithm:
(Single Source Shortest Path) SSSP (negative weights allowed)
Finding negative cycles.

```
def BellmanFord(G,u):
    init d, parent
    E = enum_edges(G)
    for i=1 .... |V|-1
        for e ∈ E:
            relax(e)
    for e ∈ E:
        if e can be relaxed:
            return ABORT
    return d, parent
```

→ found a negative cycle

Alternative: (rather than aborting)

```
for e = (v1, v2) ∈ E,
    if e relaxable,
        S.add(v1)
S' = relaxable(S)   ← bad
return d without S'
```

Runtime: $O(|V||E|)$

* correct by path relaxation lemma

Johnson's Algorithm: All Pairs Shortest Path (APSP)

1. Supernode s with weight 0 edges to all nodes.
2. Bellman-Ford from s to compute $\delta[v]$ for every $v \in V$.
3. Check for negative cycles, remove all of them. (NC's SCC)
4. Reweight each edge to be nonnegative.
   $w'(u,v) = w(u,v) + \delta_s(u) - \delta_s(v)$
5. Run Dijkstra from every node.
6. Adjust distances back: $\delta(u,v) = \delta'(u,v) - \delta_s(u) + \delta_s(v)$

Output:

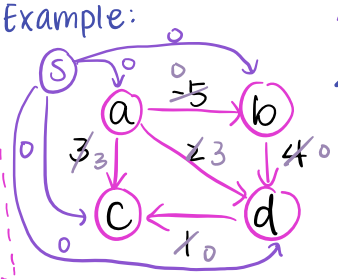
$$d[u,v] = \begin{cases} +\infty & \text{if } u \not\to v \text{ disconnected} \\ -\infty & \text{if NC between } u \to v \\ \delta(u,v) & \text{otherwise} \end{cases}$$

if NC: ① return None ② $d[u,v] = -\infty$, u to v NC; ∞ not reachable

Runtime: $O(|V|^2\log|V| + |V||E|)$
(Dijkstra for each vertex; B-F)

* Parent pointers to reconstruct shortest paths

Example:



1. Supernode s
2. B-F from s
   $\delta_s(a) = 0$
   $\delta_s(b) = -5$
   $\delta_s(c) = 0$
   $\delta_s(d) = -1$
3. No negative cycles ✓
4. Reweighing the edges
   $w'(u,v) = w(u,v) + \delta_s(u) - \delta_s(v)$
5. Run Dijkstra from every node.

	a	b	c	d
a	0	∅	∅	∅
b	∞	0	5	∅
c	∞	∞	0	∞
d	∞	∞	1	0

6. Readjust edge weights.
   $\delta'(u,v) = \delta(u,v) - \delta(u) + \delta(v)$
   $(a,b) \to 0 - 0 + (-5) = -5$
   $(a,c) \to 0 - 0 + 0 = 0$
   $(a,d) \to 0 - 0 + (-1) = -1$
   $(b,c) \to 0 + 5 = 5$
   $(b,d) \to 4$
   (undos 4)

Dynamic Programming:

DP = Recursion + memoization - recursive calls

Problems must have optimal subproblem property

1. Recursive solution
2. Structure of inputs as arguments.
3. Memoization
4. Remove recursive calls by unwinding recursion.

SRBTOT: (coin problem example)

- **S**ubproblems: Define dimensions & what each entry in memoization table represents
  ex: T(i) = max of A[i:n] for i=0...n-1
- **R**elationship: Describe how each entry depends on other entries.
  ex: $T(i) = \max(A[i] + T[i+1], T[i+1])$ (pick A[i]; don't pick A[i])
  $T(n-2) = \max(A[n-2], T[n-1])$
- **B**ase Case: Entries that don't depend on other entries
  ex: T(n-1) = A(n-1)
- **T**opological: Argue that the relations don't have a cycle in their dependency graph.
  ex: T(i) depends on entries T(>i), fill R→L.
- **O**utput: Show how output of original problem depends on entries of T.
  ex: output = T(0)
- **T**ime: Analyzing runtime. ex: n entries, $\Theta(1)$ per entry
  # of entries in T × work per entry — total: $\Theta(n)$

Path Relaxation Lemma:

Suppose given shortest path $u \to u_1 \to u_2 \to \dots \to u_k$ &
suppose we relax edges of p in order, then
$d[u_i] = \delta[u_i]$ for $\forall i$.

Proof by induction.

Upshot of lemma → Find some ordering of edges (possibly repeated) such that every shortest path is a substring ⇒ Relaxing in this order (apply lemma) gives correct shortest path.

Graph Modification Techniques:

- Duplicating entire graph & connecting * Good for BFS
  → For when some constraints or multiple states.
- Supernode (source, sink) → For connecting indistinguishable sources/destinations.
- Breaking apart edge weights to unweight them
  → for when edge weights are constant/bounded
  → lets you run DFS (linear time)
- Negating edge weights → Finding "longest paths"

6.1210 FINAL CRIB SHEET

Selinna Lin

Shortest Path & DP

DAG Relaxation If we want to go from s to v

S: $T(v) = \delta(s,v) \quad \forall v \in V$

R: Maybe this? $T(v) = \delta(s,v) = \min\{w(s,u) + \delta(u,v)\}$ (don't have this in table!!) } Think of this as trying to leave s.

This is better → $T(v) = \delta(s,v) = \min\{\delta(s,u) + w(u,v)\}$ } Think of this as trying to approach v.

B: $T(v) = \infty$ if node v has in-degree 0

$T(s) = 0$

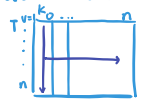
T: topological sort of DAG

O: All of T

Runtime: $O(\sum_v (1 + \text{in-degree}(v)))$ (base case, if in-degree = 0; takes in-degree(v) time to process a vertex v.)

Bellman Ford: $T(v,k) = \delta_k(s,v)$ ($k = 0, \dots, V$) weight of s.p. from $s \to v$ using at most k edges

$T(v,k) = \min\begin{cases} T(v,k-1) \\ \min_{v \in Adj^-} T(v,k-1) + w(u,v) \end{cases}$ ($k = 1, \dots, V$)



Also takes $O(|V|+|E|)$ time to find topological sort.

Total Runtime: $O(|V|+|E|)$

Floyd-Warshall As good as Johnson for dense graphs, but worse for sparse ($|E| = O(|V|)$)

S: $T(u,v,k)$:= distance from u to v using only nodes in $\mathcal{U}_k$. $\forall u,v \in V$ and $k = 0, \dots, n$.

R:

$$T(u,v,k) = \min\begin{cases} T(u,v,k-1) & \text{not using } k \\ T(u,k,k-1) + T(k,v,k-1) & \text{using } k \end{cases}$$

B: $k = 0 \quad T(u,u,0) = 0 \quad T(u,v,0) = \begin{cases} w(u,v) \text{ if } \exists \text{ edge } (u,v) \\ \infty \text{ otherwise} \end{cases}$

T: Entries depend on smaller values of k. Fill T from $k = 0 \to n$.

O: $T(u,v,n)$ is distance from u to v.

Runtime: $\Theta(|V|^3)$ $|T| = O(n^3)$, $n = |V|$ and each entry takes $\Theta(1)$.

Greedy Algorithm

Activity Scheduling Problem: Earliest due dates

- Pick Greedy Choice
- Greedy Choice Property (GCP): There $\exists$ optimal solution that includes (greedy choice)

  Typical proof methods → Consider arbitrary optimal solution. Change it to contain greedy choice, while still being an optimal solution.

  → Show that anything but the greedy choice is worse.
- Algorithm + Correctness → Proof usually by strong induction (take away the greedy choice, by assumption, algorithm solves it)
- Runtime

Huffman Codes Tree going left: 0, right: 1. Runtime: $O(n \log n)$ Build tree bottom up.

Greedy Choice: 2 smallest frequencies are siblings in tree.

Subset Sum (all ≥ 0) Input: $A = [a_1, a_2, \dots, a_k]$ L

Goal: Decide if $\exists$ subset of A that sums to L.

S: $T(i,t)$:= T/F if there $\exists$ a subset of $A[i:]$ with sum t

R:

$$T(i,t) = \begin{cases} \text{True if } \exists\ T(i+1,t) \text{ that is True or} \\ \qquad T(i+1, t-A[i]) \text{ and } t \geq A[i] \\ \text{False otherwise} \end{cases}$$

B: $T(i,0) = \text{True}$

$T(n,t) = \text{True IFF } A[n] = t \text{ OR } t = 0$

T: $T(i,t)$ depends on $T(u,v)$ s.t. $u > i$

O: $T(0,L)$

Runtime: $O(k(L+1)) = O(Ln)$ (n: # of elements in A.)

Note: Not polynomial runtime, since we can input a really big L.

Polynomial Time Algorithms:

BFS, DFS, Dijkstra, Bellman Ford, Merge Sort, Insertion Sort.

Pseudo-polynomial Time:

- Runtime is polynomial in n (input size) and x (max of one of the input vals)
- $O(n^c)$ if largest numerical val in an input is $O(n^{c'})$

Ex: selection sort, $O(n+u)$ counting sort, $O(n \log_n u)$ radix sort, DAA sort

Brute Force Algorithms:

Going through all permutations

$O(2^n)$ Ex: for all subsets of A:

if sum(subset) = e : return True

Brute Force search is best algorithm for P ≠ NP Problems.

Selinna Lin

Complexity: $P \subseteq NP \subseteq PSPACE \subseteq EXP$ (PSPACE: algorithms using $O(n^c)$ memory)

Note: Every problem reducible to itself ($A \leq_P A$)

NP-Hard → B is NP-hard if $\forall A, A \in NP, A \leq_P B$. (Everything in NP can be reduced to B)

NP-Complete → B is NP-complete if B is NP-hard and $B \in NP$.

Ex: Knapsack, subset sum, 3 coloring

If any NP-Complete problem has a poly time algorithm, then all of them would!

NP: {problems s.t. ∃ "verification algorithm" V that runs in polynomial time ($O(n^c)$)}

V is given instance x (length n) & certificate c (length $O(n^c)$)

x = T/F Question c = solution V = grader

↑ can be verified in polynomial time

If ans(X) = YES → ∃ c s.t. V(x, c) = Yes (There exists some c_x s.t. $V_A(x, c_x)$ outputs yes.)

If ans(X) = NO → ∀ c s.t. V(x, c) = No (For all c, $V_A(x, c)$ outputs no.)

Reduction: Computational problem A reduces to B if ability to solve B efficiently → solve A efficiently.

- $A \leq_P B$ (A reduces to B in poly time) if poly time alg. R s.t. given instance x in A, output an instance y = R(x) in B w/ same solution.
- B is at least as hard as A.

Activity Scheduling Problem:

activity $a = (t_i, d_i)$ (duration, due date)

if a is scheduled at time t

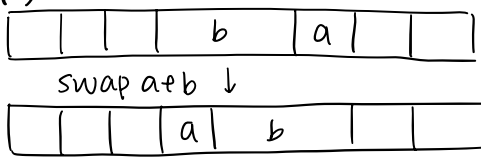
lateness → $l(a) = \max(0, t + t_i - d_i)$

Goal: Schedule activities s.t. we minimize the worst lateness.

GCP: ∃ an optimal schedule that schedules activities with the earliest due dates.

Proof: (for our GCP)

Let's start with S:



S':

$l(x)$: lateness of x using schedule S

$l'(x)$: lateness of x using schedule S'

Want to prove: $\max_a l'(a) \leq \max_a l(a)$

$\max_a l'(a)$: worst lateness in S'

$\max_a l(a)$: worst lateness in S

Assuming that worst lateness comes from a. claim → $l'(a) < l(a)$ ✓

(If you start activity before, it'll be late less.)

a has earlier due date.

Definition of greedy choice → Since a is the greedy choice, $l'(b) < l(a)$.

Under S', b finishes at the same time a does under S. (b unders')

activity due earliest → late more, so $l'(b) < l(a)$ ← a unders

Miscellaneous:

* Careful to not assume things in problems are sorted already. (especially for Greedy)

- x numbers can't be sorted with y comparisons $x!$ vs. 2^y
- Hash function maps keys to $\{1, \ldots, m\}$ (m = size of table)
- Graph w/ forward edges can't be a tree.
- Huffman Code → prefix-free (✓), optimal → add together (frequency of letter x bits to encode it)

  ex: aabbbccccddddd a: 011, b: 010, c: 00, d: 1

  $2 \times 3 + 3 \times 3 + 4 \times 2 + 5 \times 1 = 28$

  ① make graph & find optimal #

  ② check if encodings optimal
- Set AVL tree: keys in order (things in order of value)
- Sequence AVL tree: elements in order (things in order of index)
- Linked list: Can only move from one node to next (or take head pointer)

Sequences	Operations $O(\cdot)$				
	Container	Static	Dynamic		
	`build(X)`	`get_at(i)` `set_at(i,x)`	`insert_first(x)` `delete_first()`	`insert_last(x)` `delete_last()`	`insert_at(i, x)` `delete_at(i)`
Array	n	1	n	n	n
Double Linked List	n	n	1	1	n
Dynamic Array	n	1	$1_{(am)}$	$1_{(am)}$	n
AVL-trees	n	$\log n$	$\log n$	$\log n$	$\log n$

Sets	Operations $O(\cdot)$				
	Container	Static	Dynamic	Order	
	`build(X)`	`find(k)`	`insert(x)` `delete(k)`	`find_min()` `find_max()`	`find_prev(k)` `find_next(k)`
Array	n	n	n	n	n
Sorted Array	$n \log n$	$\log n$	n	1	$\log n$
Direct Access Array	u	1	1	u	u
Hash Table	$n_{(avg)}$	$1_{(avg)}$	$1_{(am)(avg)}$	n	n
AVL-trees	$n \log n$	$\log n$	$\log n$	$\log n$	$\log n$

Priority Queues	Operations $O(\cdot)$			Priority Queue Sort	
	`build(A)`	`insert(x)`	`delete_max()`	Time	In-place?
Dynamic Array	n	$1_{(am)}$	n	n^2	Y
Sorted Dynamic Array	$n \log n$	n	$1_{(am)}$	n^2	Y
Balanced Binary Tree	$n \log n$	$\log n$	$\log n$	$n \log n$	N
Heaps	n	$\log n_{(am)}$	$\log n_{(am)}$	$n \log n$	Y

Master Theorem for $T(n) = aT(n/b) + \Theta(n^c)$

case	solution	conditions
1	$T(n) = \Theta(n^{\log_b a})$	$c < \log_b a$
2	$T(n) = \Theta(n^c \log n)$	$c = \log_b a$
3	$T(n) = \Theta(n^c)$	$c > \log_b a$

Restrictions		**SSSP Algorithm**		
Graph	Weights	Name	Running Time $O(\cdot)$	How it works
DAG	Any	DAG Relaxation	$\lvert V\rvert + \lvert E\rvert$	Relax in topological order
General	Unweighted	BFS	$\lvert V\rvert + \lvert E\rvert$	Relax level by level
General	Nonnegative	Dijkstra	$\lvert V\rvert \log \lvert V\rvert + \lvert E\rvert$	Relax in priority order
General	Any	Bellman-Ford	$\lvert V\rvert\lvert E\rvert$	Relax in $\lvert V\rvert$ rounds